

Web Crawling and Indexes

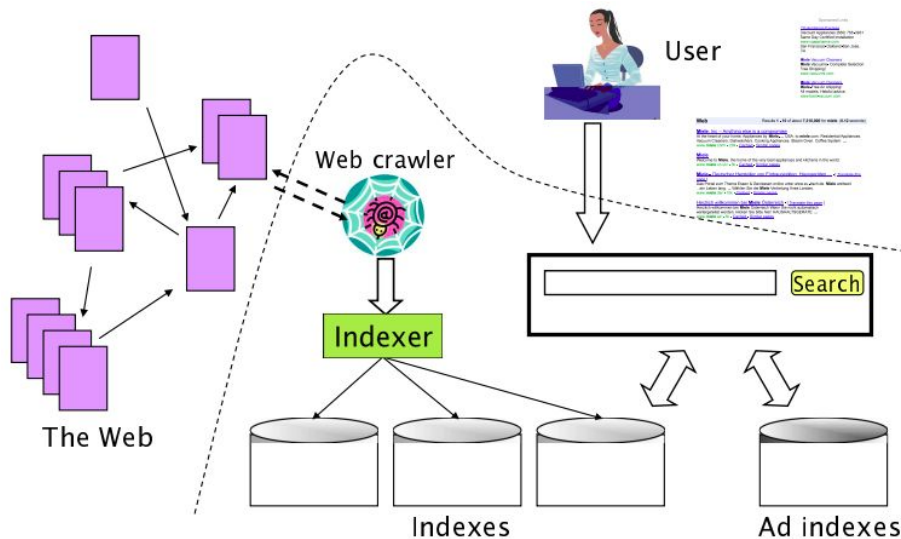
Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Overview

- **Web Crawling** is the process by which we gather pages from the web in order to index them and support a Web Engine
- Web Crawler (also named **Spider**) is in charge of this job
- The complexity of the web makes such a job **very complex**



Features a crawler MUST provide

- Robustness: some Web Server might generate ***spider trap***, which are pages generated to mislead crawlers into getting stuck fetching an infinite number of pages in a particular domain. Not all traps are malicious, some of them are side effects of faults made by developers. Crawlers must **be resilient** to such traps;
- Politeness: Web servers have implicit and explicit policies **regulating the rate** at which a crawler can visit them (not too often, not too late). These policies must be respected

Features a crawler SHOULD provide

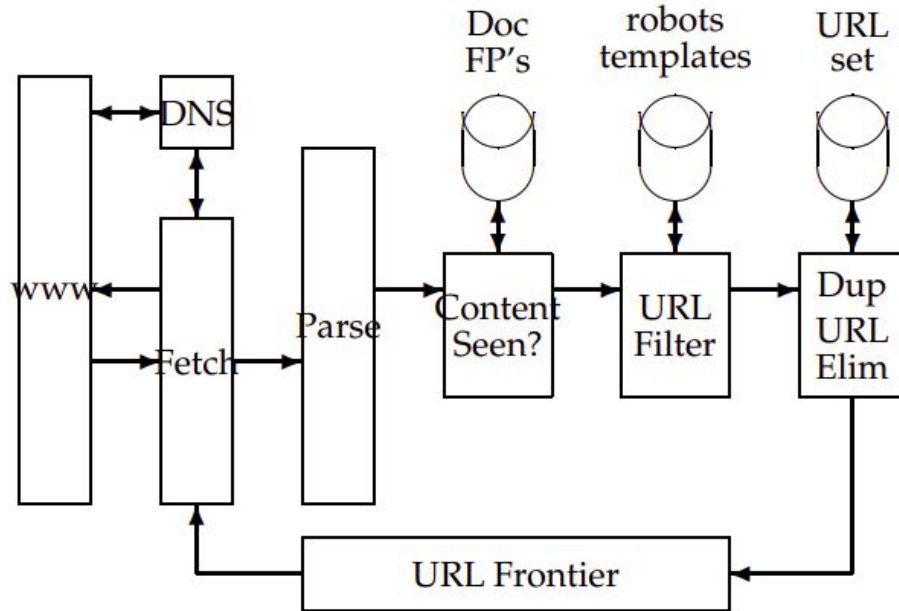
- Distributed: crawlers should be able to run gathering web pages in a distribute fashion
- Scalable: the crawler architecture should permit scaling up, by adding extra machines
- Performance and Efficiency: the crawler system should use efficiently all the resource such as bandwidth, processor, storage, ecc.

Features a crawler SHOULD provide

- Quality: the crawler should start **fetching useful pages first**
- Freshness: a crawler should gather pages with a frequency that approximate **the rate of change of the page**
- Extensible: the crawler should be easily extensible to work properly even with new data format, new fetch protocols. This demands that the crawler architecture be **modular**

The Crawler Architecture

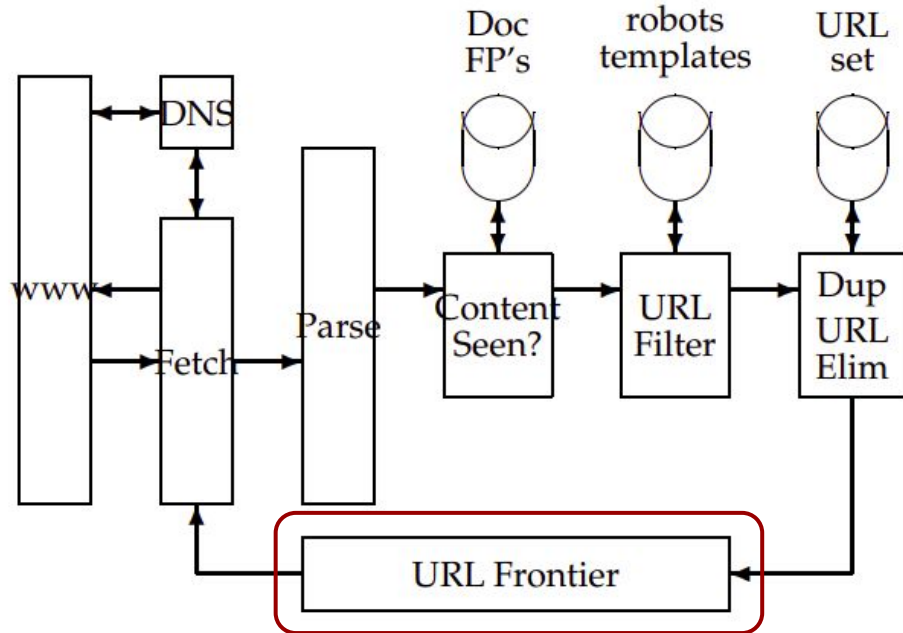
- URL Frontier
- DNS Resolution module
- Fetch module
- Parsing Module
- Duplicate Elimination Module



How a crawler works

The crawler owns a “**seed set**”, a small static list of possible websites to crawl

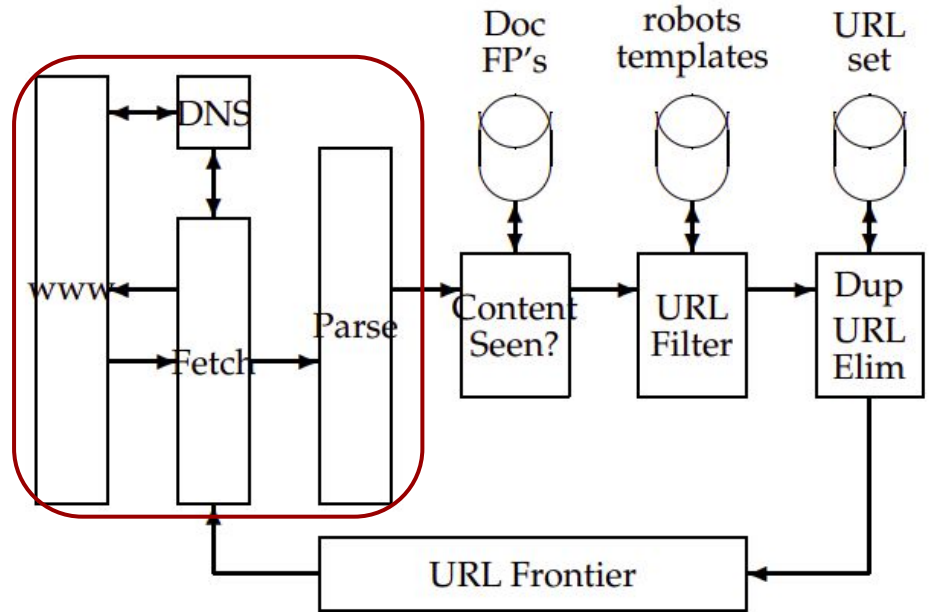
The seed set is then stored in the **URL Frontier**, representing all URL to be fetched



How a crawler works

The Crawler Thread takes a URL from the URL Frontier and fetch the web pages using HTTP protocol

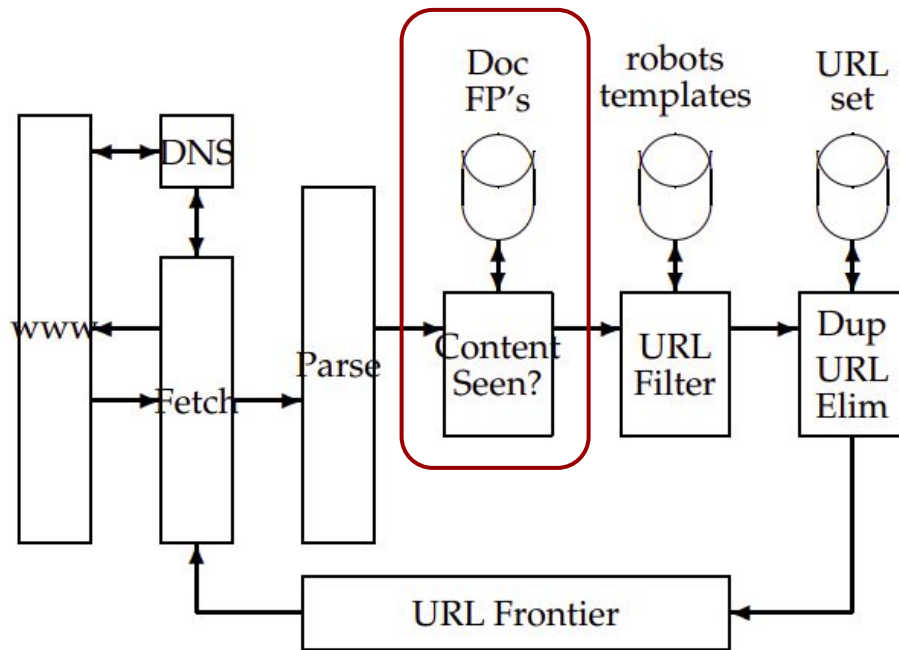
The page is stored in a temporary area, and then it is **parsed**. Text and Links are extracted and sent to the **Indexer**



How a crawler works

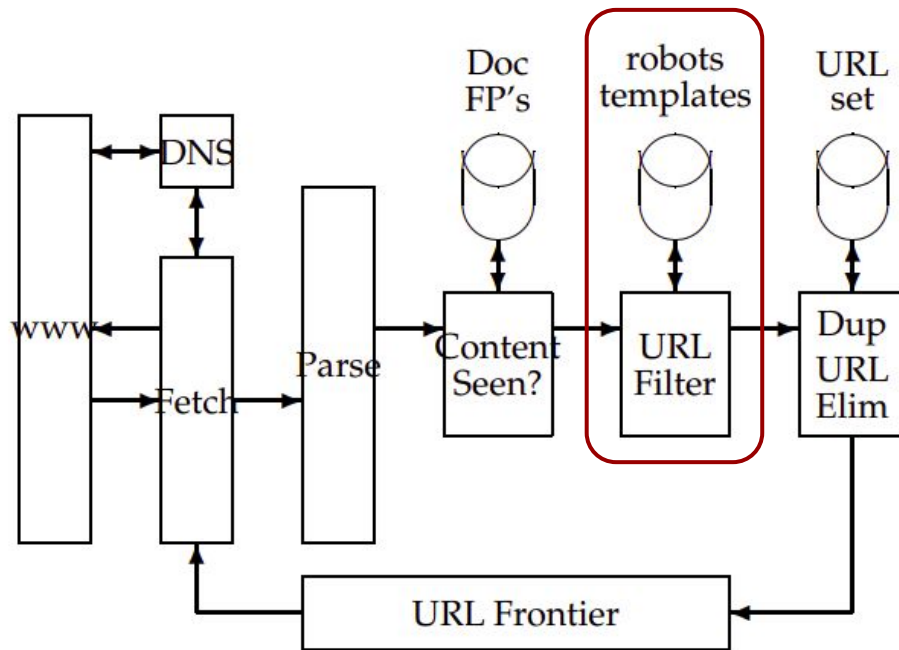
Text is analyzed to test whether a web page is similar to another already indexed (Doc FP's contains all the Fingerprint for all web pages already indexed)

Some implementations, instead of using **Fingerprint**, make use of **Shingles**



How a crawler works

All links are also tested to determine if they should be added to the URL frontier. The URL Filter is used to determine whether the extracted URL should be excluded from the frontier based on one of several tests. It depends on the results of **several rules** (see later)



How a crawler works

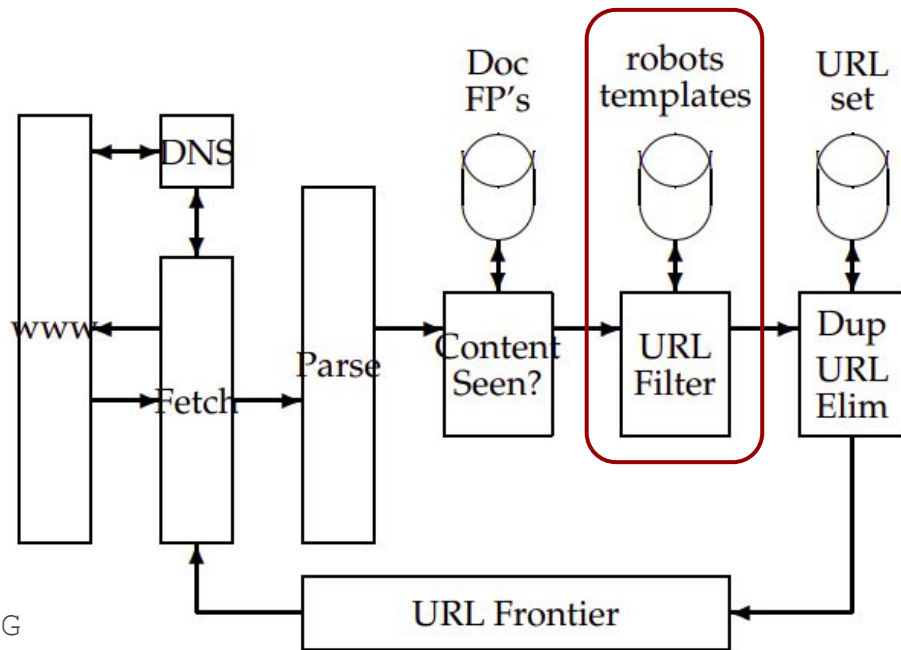
Once the URL has been selected to be put into the URL Frontiers, the link is **normalized**

For example, the following tag

```
<a href=
"/wiki/Wikipedia:General_disclaimer"
title="Wikipedia:General
disclaimer">Disclaimers</a>
```

is normalized in

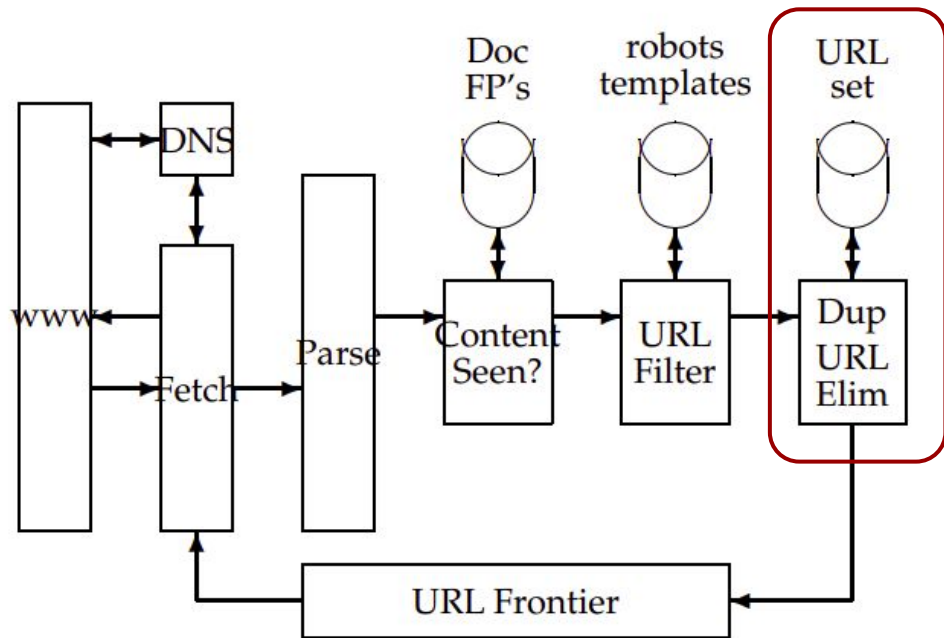
```
http://en.wikipedia.org/wiki/Wikipedia:G
eneral_disclaimer
```



How a crawler works

The Duplicate URL Elimination Module is in charge of verify if an URL is already in the URL Frontier or not. If it is, the URL is discard. Otherwise, it is added.

Being the number of possible URL too high, **an efficient way of detecting duplicate URL** should be used.

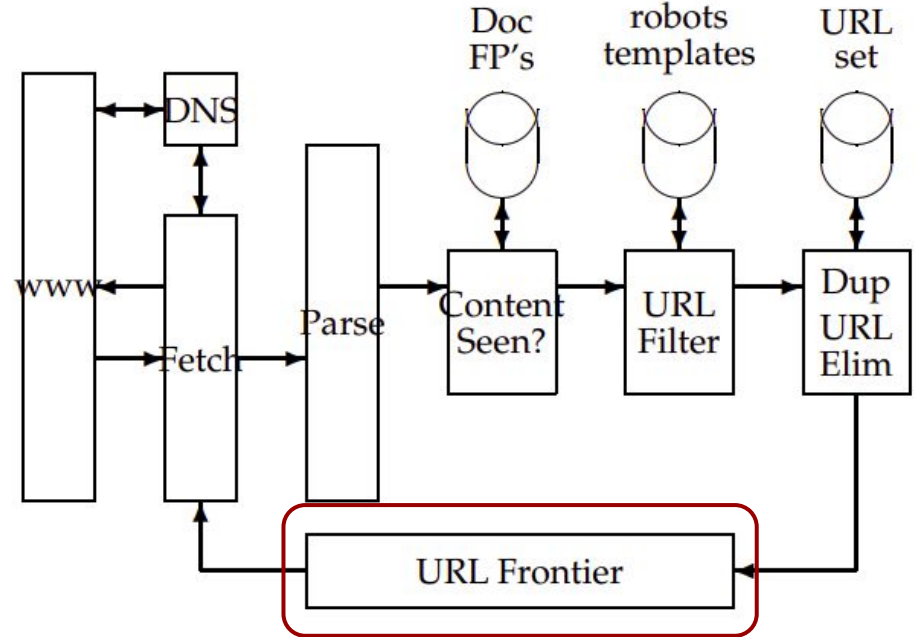


How a crawler works

Finally the link is put into the URL Frontier only if the same normalized link has not been already **stored into the URL Frontier**

When the link is stored in the URL Frontier, a **priority** is assigned.

Links having an **higher priority** will be fetched before those having a lower priority



Filtering URL

As we saw previously, the URL Filter is a component of a Crawling system which is used to determine whether the extracted URL should be excluded from the frontier based on rules.

Rules can be:

- **domain-based rules:** for example all .com domain
- **robots-based rules:** some web pages can be kept off-limit to crawling. This can be done making the file robots.txt available to the search engine

Robot Exclusion Protocol

```
User-agent: *  
Disallow: /yoursite/temp/
```

```
User-agent: searchengine  
Disallow:
```

In this example we do not allow any robot to visit any page under /yoursite/temp, except for the robot called "searchengine"

Multithreading

Crawling systems are usually developed in **multithreading or multiprocessing** fashion, allowing them to work even in distributed system (shared memory or distributed memory).

One or more threads or processes implement each component

But there are also some dedicated threads/processes

- logging
- statistics
- URL frontier snapshot (for crawler failure)

Distributing the crawler

Modern Crawling Systems are also **geographically distributed**.

How the whole system can manage fetching operations?

URL are scattered among all distributed processes

The rules for splitting are different:

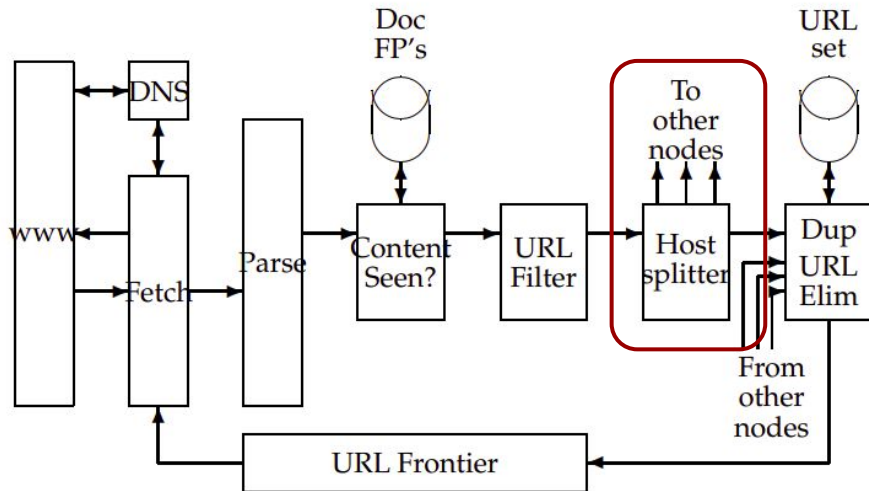
- domain based
- URL hashing

Distributing the crawler

A new component is added: **Host Splitter**

The output of the Host Splitter goes to the Duplicate URL Eliminator block of the local system or in the remote nodes

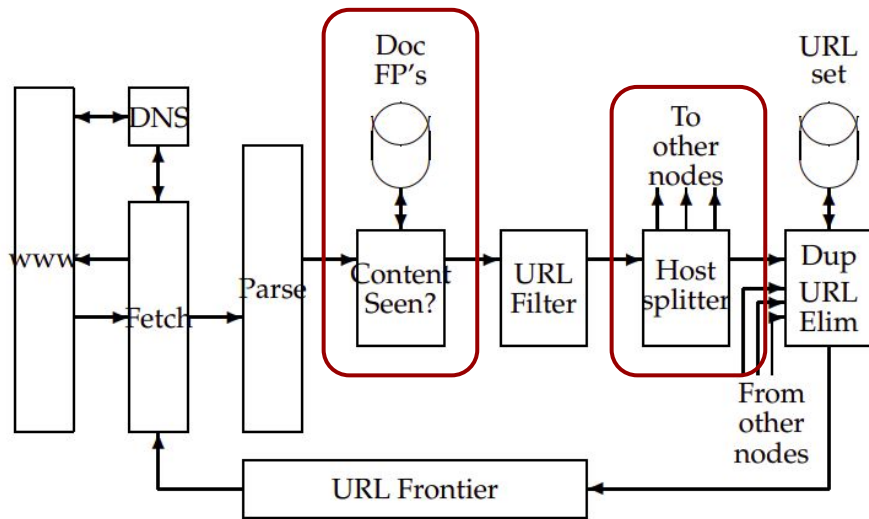
Communication between nodes happens usually using **JSON** and **REST WS**



Distributing the crawler

Introducing the Host Splitter module makes the module “Content Seen” more complex

The same content might appear **on different web servers managed by different remote threads**



URL Frontier

URL Frontier contains all the URL that should be still processed

The next URL to be processed **depends on the priority** index assigned to each URL

The priority index is computed as a combination of two factors:

- pages that change frequently should be prioritized for frequent crawling
- high quality pages

URL Frontier

Building a URL frontier just as a priority queue does not work.

Let's think about a web page having several URL links to the same host. All of them will have the same priority.

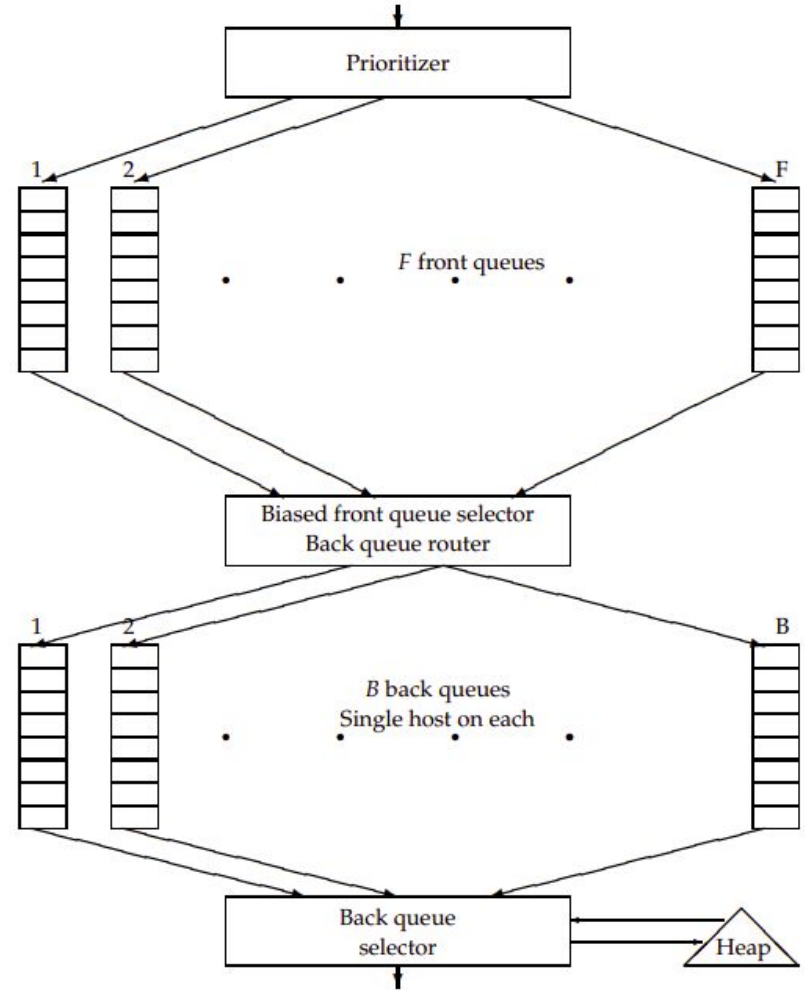
This might result in a **burst of fetch requests** to the same host. This violates the **politeness principle**

Solution: insert a **gap between successive fetch** requests to a host that is larger than the time taken for the most recent fetch

URL Frontier

Aims of this implementation:

1. only one connection is open at a time to any host
2. a waiting time of a few seconds occurs between successive requests to a host
3. high-priority pages are crawled preferentially



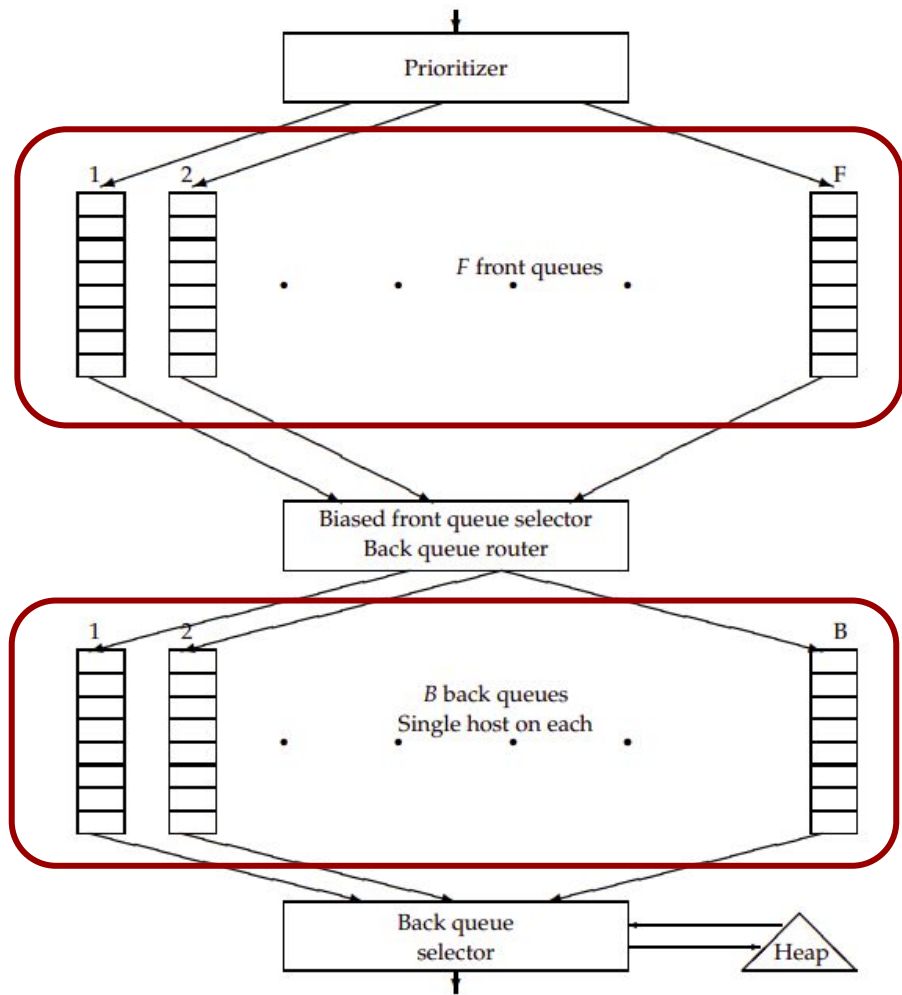
URL Frontier

Two Modules

1. F Front Queues (for prioritization)
2. B Back Queues (for politeness)

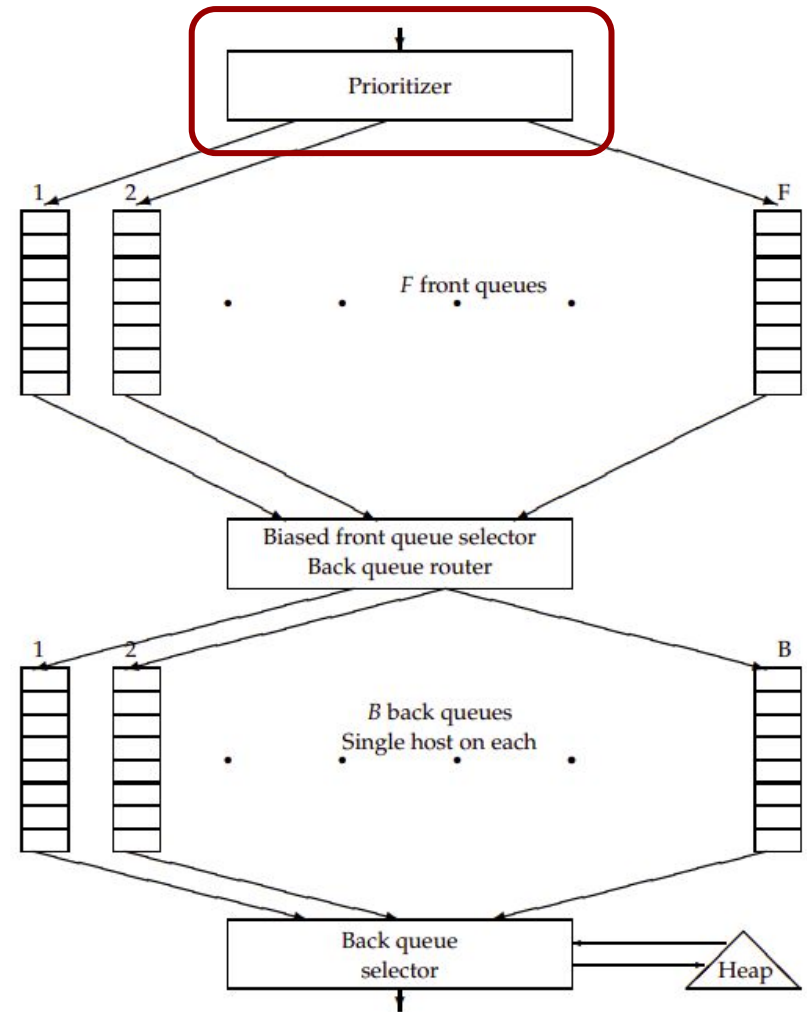
The **value of B and F** depends on the implementation

Both are **FIFO queues**



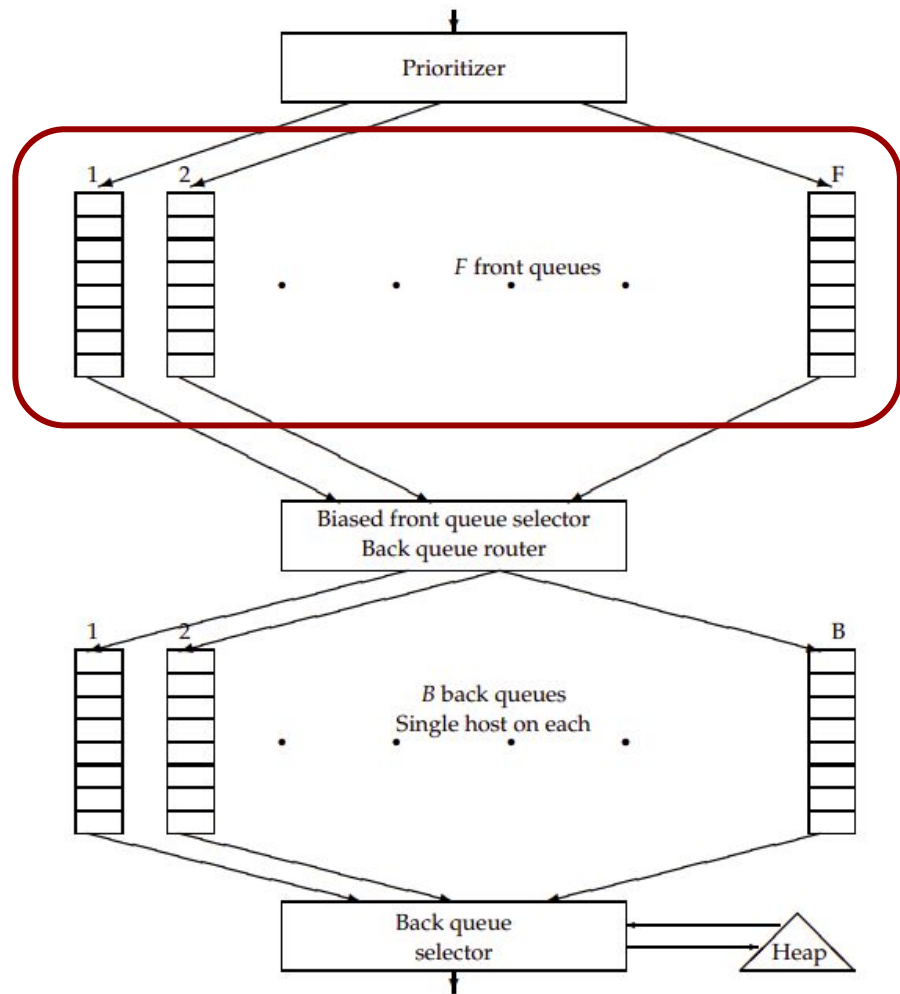
URL Frontier

- Each URL is assigned to an integer
- The priority index ranges between 1 and F
- The priority depends mainly from the **frequency of updates**
- Other heuristics can be applied (for example, if the URL refers to a **news** web site, the priority is always high)



URL Frontier

- When a URL is assigned to the index i , it is queued in the Front Queue i
- A queue can contains URL referring to **different hosts** (but all of them having the **same priority**)

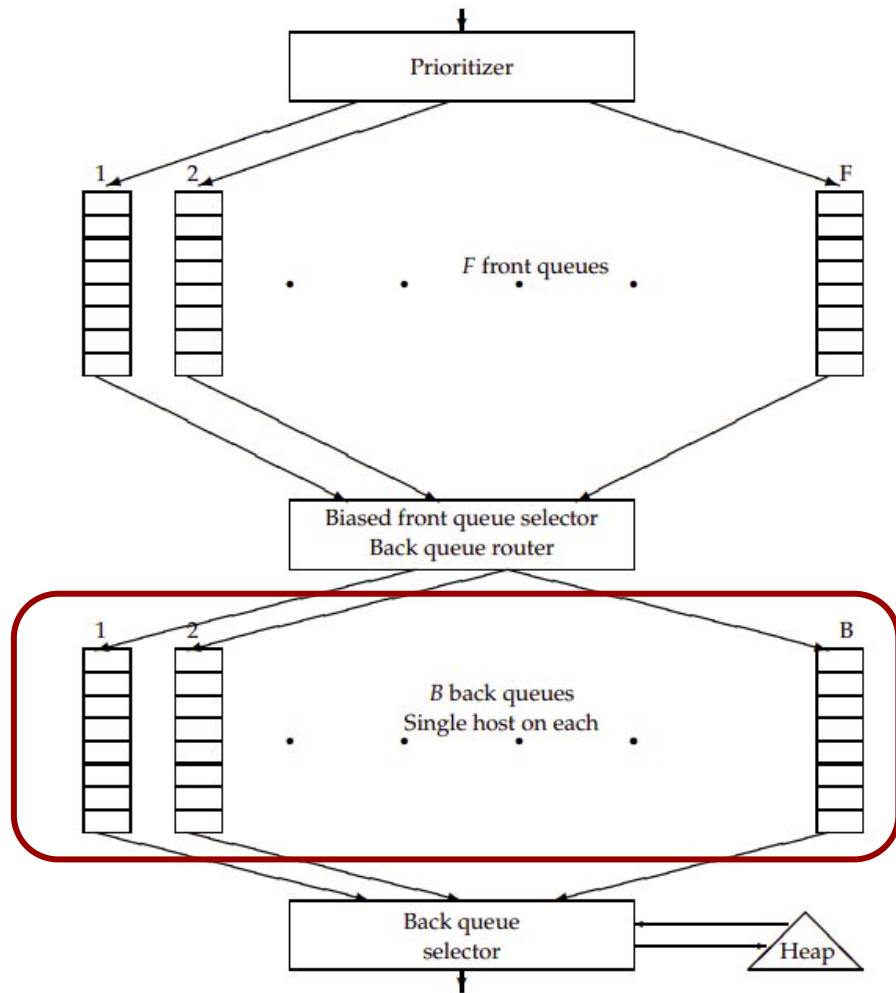


URL Frontier

The B queues maintain **two invariants**:

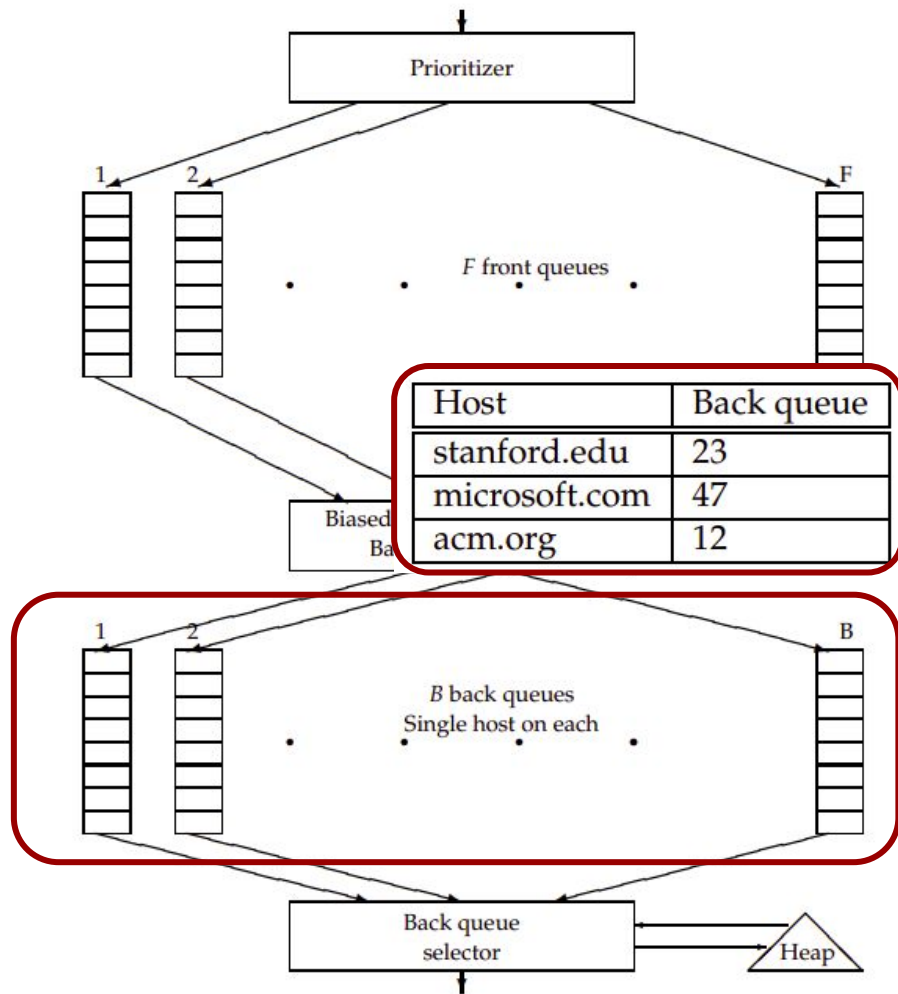
- all B queues are always non empty
- all B queues contain only links from the same host

Note that the value of B is much **smaller** than the **number of hosts**, but this is not a problem.



URL Frontier

An **auxiliary table** for mapping hosts to the Back Queues must be managed. By looking up the host into the mapping table, it would be possible to identify the queue containing all URLs referring to that host



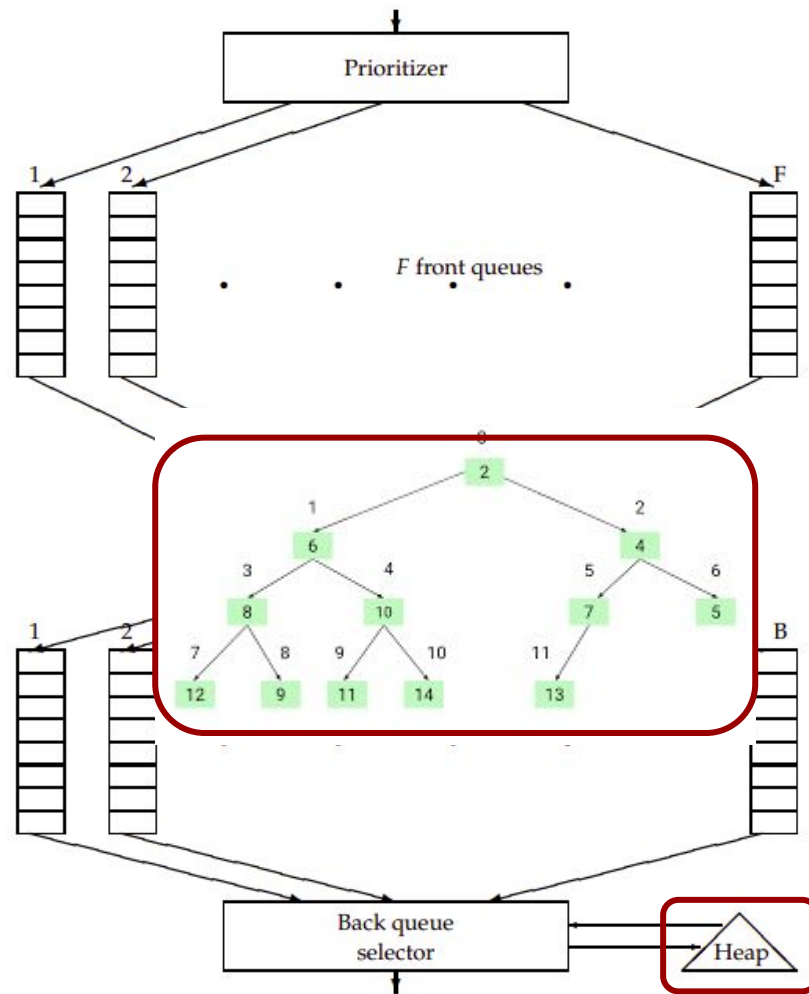
URL Frontier

There is also a **min-heap** with an entry for each back queue.

It is implemented like a binary tree

Each node contains at least two values:

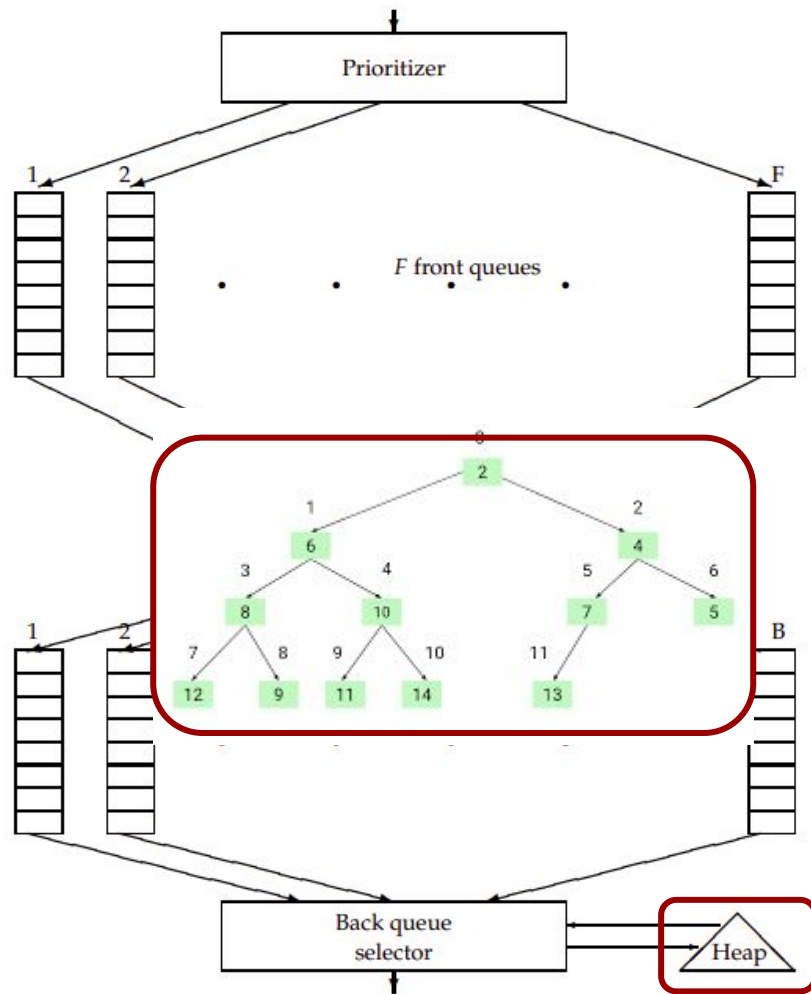
1. the corresponding queue represented by the node
2. the time when the host should be contacted back again



URL Frontier

The **heap is sorted** depending on the time the host can be contacted again.

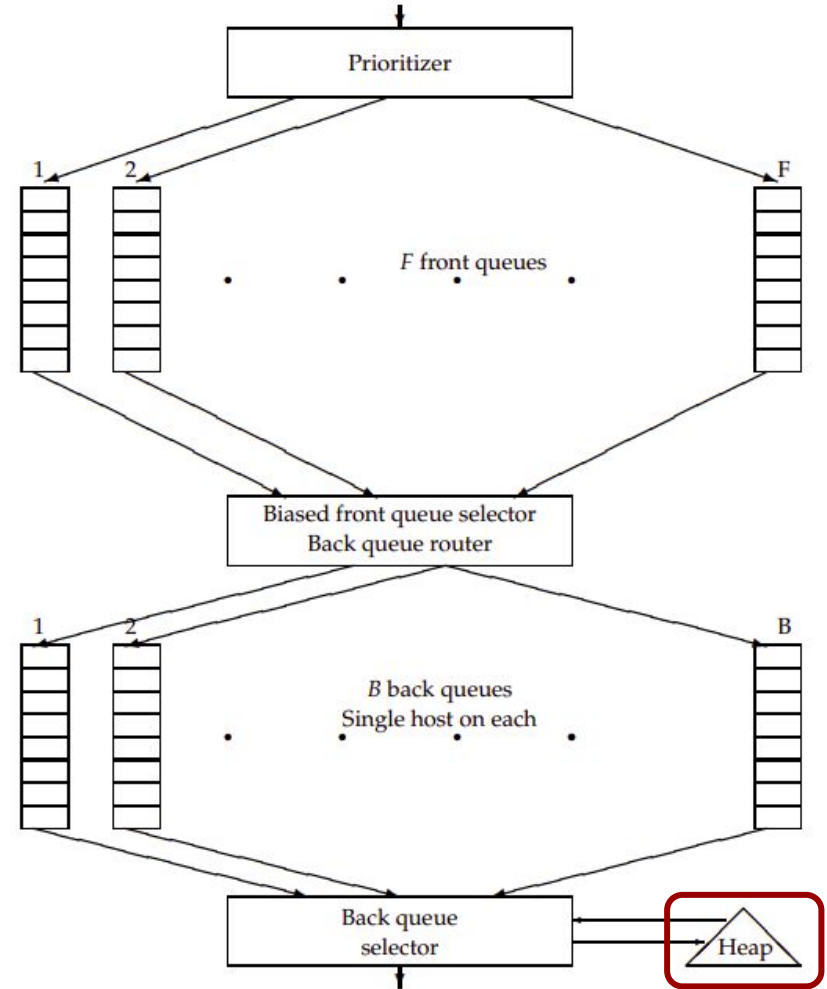
Being a min-heap, the root of the heap refers to **the host to be contacted the first**.



URL Frontier

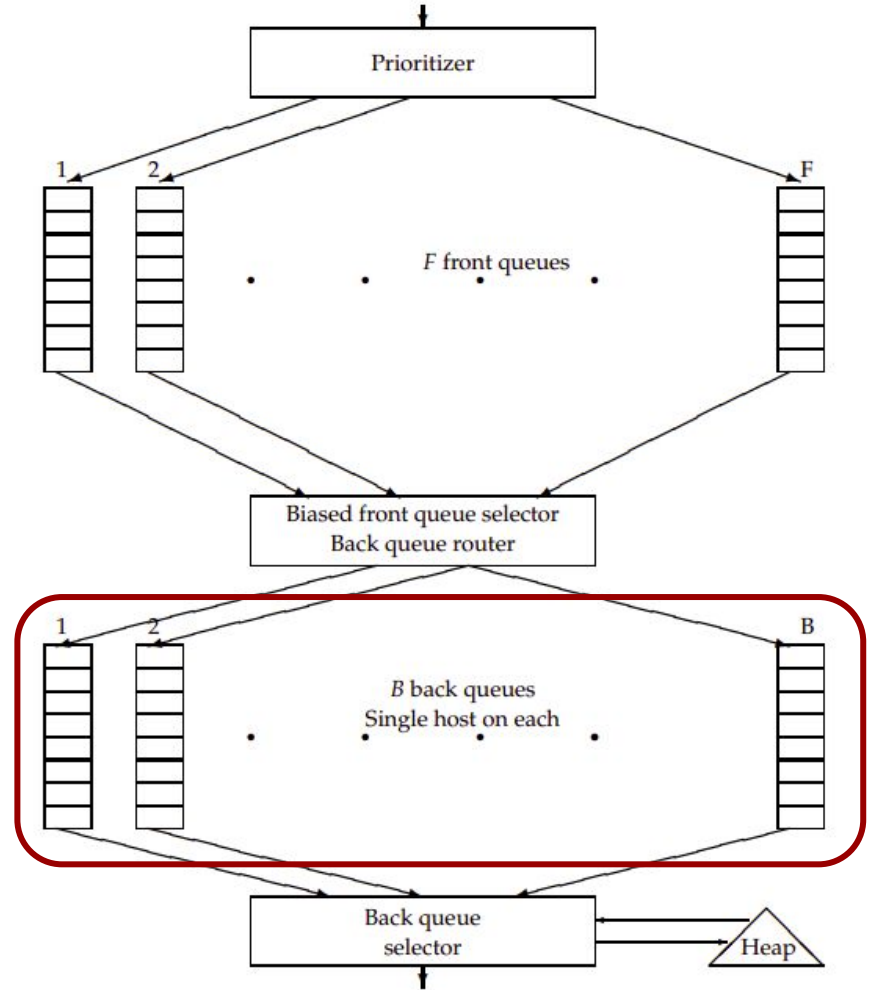
Now, let's see how URL are moved within the URL Frontier:

1. Moving URL is in charge of crawler threads
2. A crawler thread needs the next URL to be crawled, so it extracts the root of the heap
3. The heap is rearranged such that the heap root will be the next host to be contacted



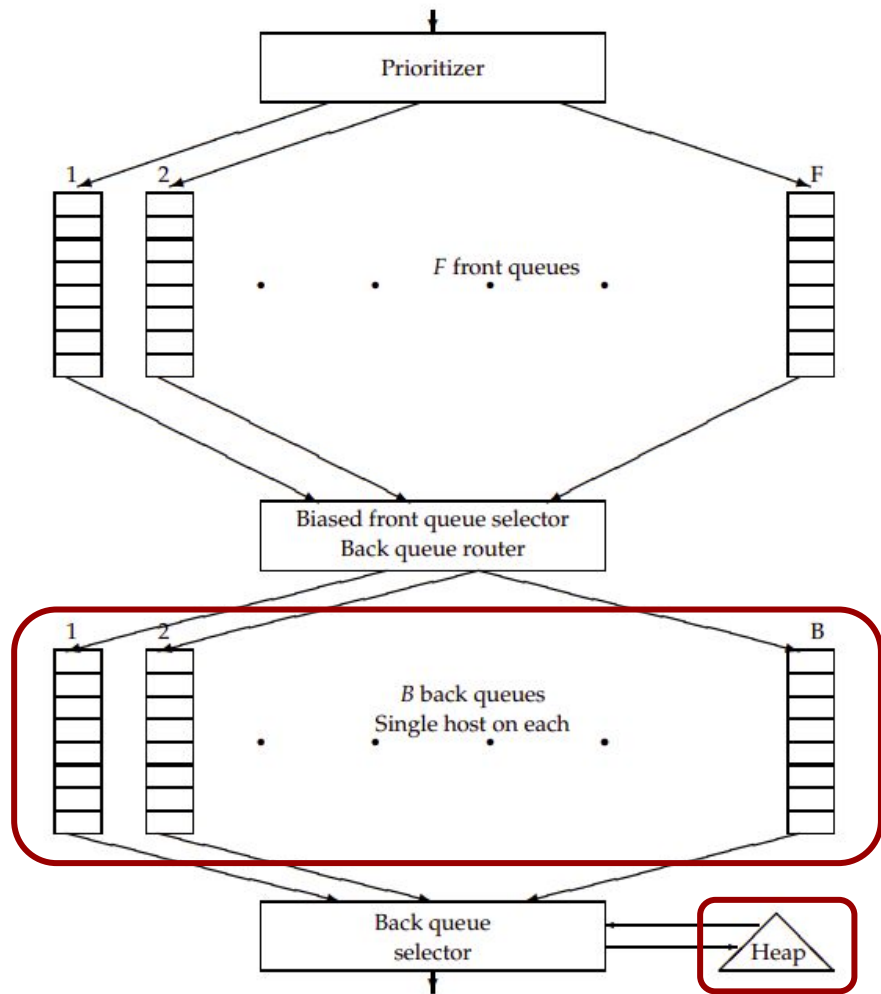
URL Frontier

4. If the time to contact the host is in the future, the crawler thread waits
5. When the time is ready, the crawler thread retrieves from the back queue b represented by the heap root the next URL u to fetch
6. The URL u in the head of the back queue b is then fetched



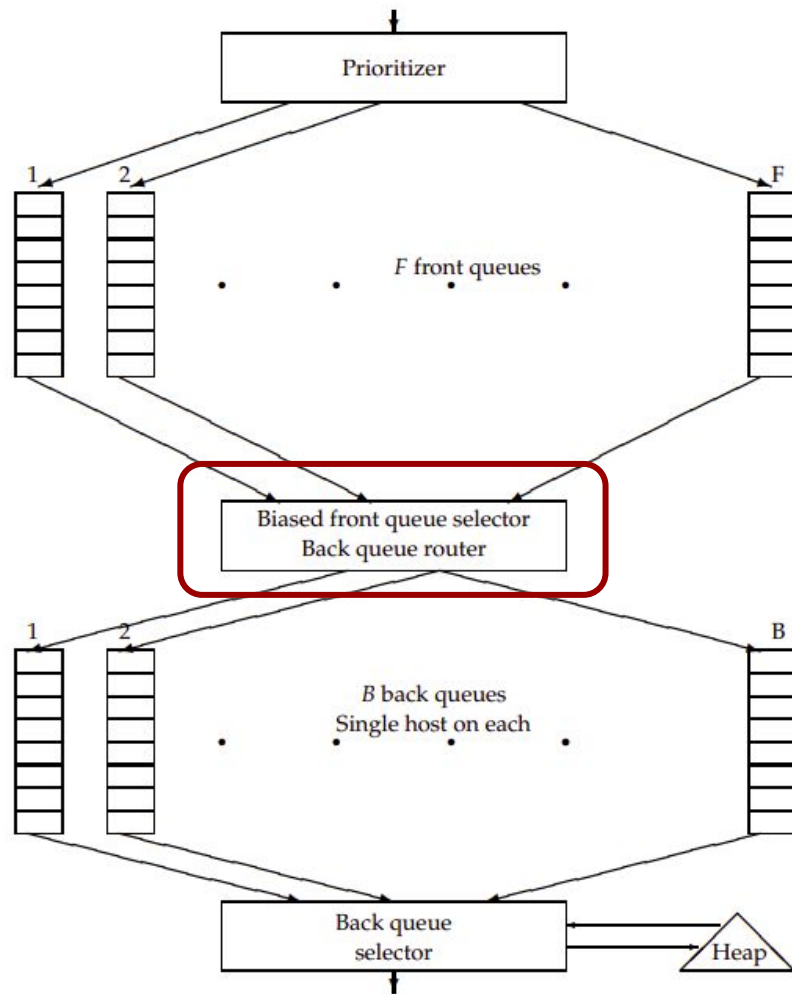
URL Frontier

7. After fetching u , the crawler thread verifies if the back queue b is empty.
- If not empty: the back queue b is again inserted in the heap with a new time to visit
 - if empty:
 - the auxiliary table is updated (by removing the correspondence of the URL with the Back Queue ID)
 - the Crawler Thread selects the index of the Front Queue f from which to retrieve the URL.



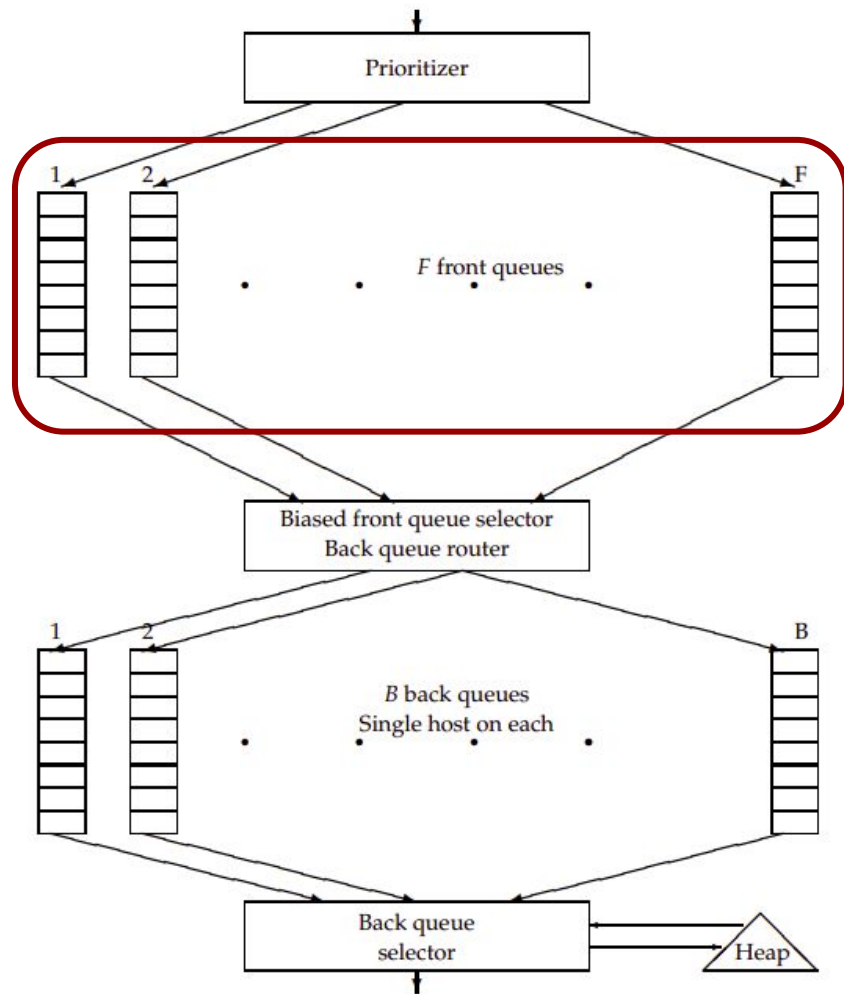
URL Frontier

The selection of the Front Queue should ensure that URLs of high priority are **selected more often** than those having low priority.



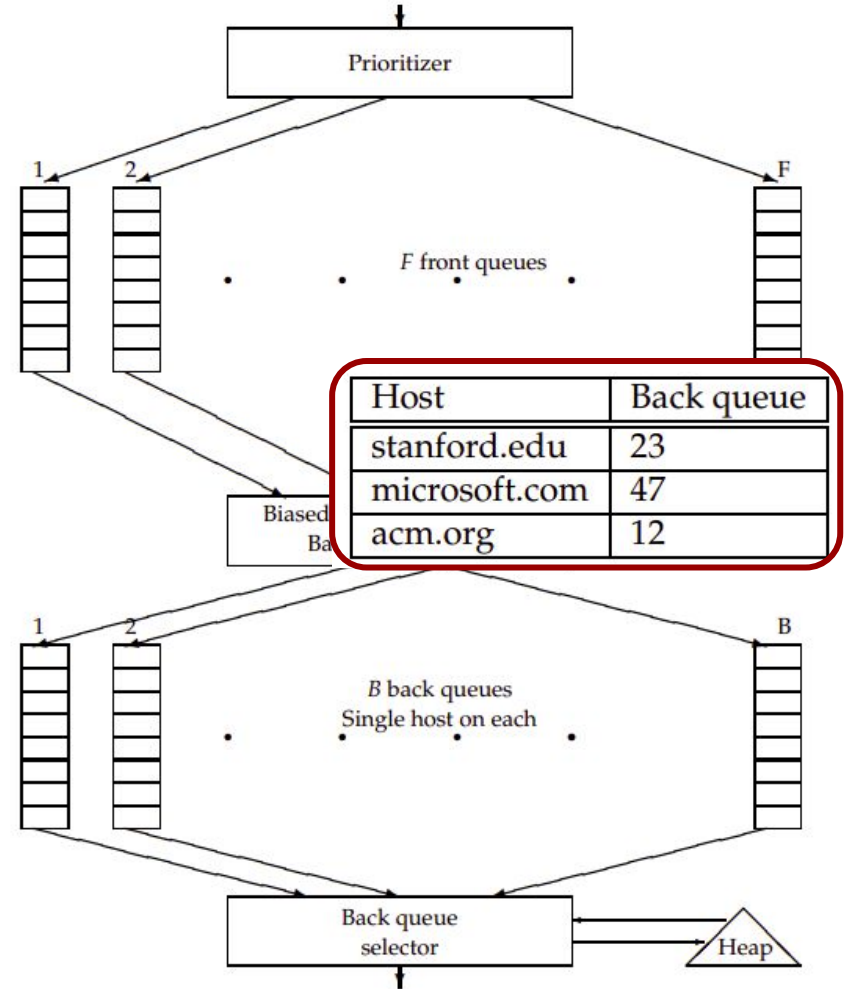
URL Frontier

8. Once the Front Queue f has been selected, the crawler process extract the URL v in the head of Front Queue f



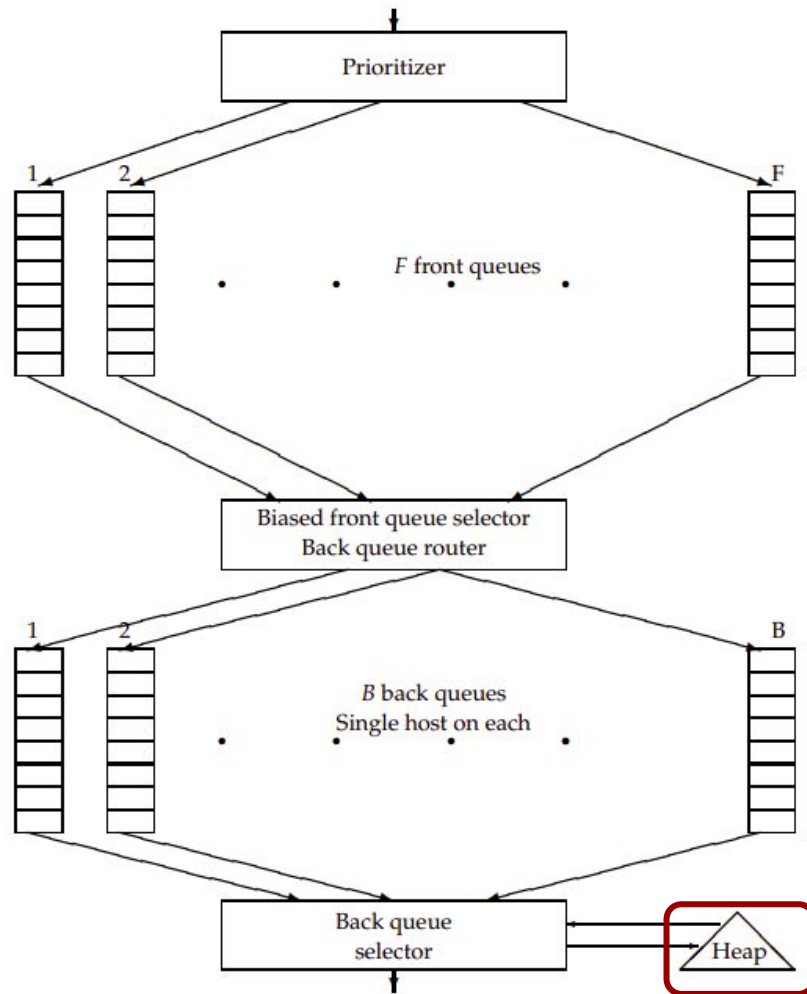
URL Frontier

9. By looking at the mapping table, the Crawler Thread verifies whether the URL v is already in a Back Queue.
 - a. if yes: v is added to the back queue and a new Front End Queue is selected for the next candidate URL to be inserted into the empty back queue b .
 - b. if not: v is added to the empty Back Queue b and the mapping table is updated



URL Frontier

10. If v is added to the Empty Back Queue j , the Crawler Thread inserts a new entry on the Heap for j with a new earliest time.
11. The Crawler Thread continues with its processing



URL Frontier

Some details:

- the number of Front Queues, the heuristics of assigning priorities and the policy for selecting the next Front Queue determine the **priority properties** of our implementation
- Recommendation suggests to use a number of Back Queues **three times higher** than the number of Crawler Threads
- URL frontier may grow and require more memory than that available on the node. The solution is to let most of the **URL frontier reside on disk** and moving into the memory just a small portion